LAB 2 Parity Bit

Student: Alonso Emmanuel López Macías
Email's: usuario_cocyten_32_x@cinvestav.mx
alemlopezma@gmail.com

Student: Cesar Eduardo Inda Ceniceros
Email's: usuario_cocyten_32_x@cinvestav.mx
cesar.inda@iteso.mx

Full Professor: Dr. Martin Gonzalez Perez

09/04/2025

CINVESTAV, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.
COCYTEN, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.
Technical Report

Abstract: *FPGA Terasic Board – ALTERA Cyclone IV EP4CE115F29C7N*
IDE – Quartus Prime 18.1
Boolean Algebra
Digital Gates & Logical Comparators
Git & GitHub Control Version

Keywords: *Parity Bit, Combinational Logic, Even Parity, Odd Parity & Bit Word.*

Table of Contents

Table of Figures

1.- INTRODUCTION

In this practice, an implementation of an 8-bit parity bit was carried out. This bit is added to the data frame during transmission and serves as a mechanism to detect whether the added bit is correct or incorrect, helping to identify errors. In a data transmission—such as between two FPGAs or between an FPGA and another system—the transmitter calculates the parity bit and sends it along with the data. The receiver then recalculates the parity of the received data and compares it with the received parity bit. If they do not match, an error has occurred (at least one bit has changed). The parity bit can only detect single-bit errors; it cannot correct them or detect all multiple-bit errors. It is also used when storing data in memory (RAM, registers, buffers), where the data is stored along with the parity bit. Upon retrieval, the parity is verified, and if an error is found, the data is discarded, or a retransmission is requested.

1.1.- Requirements

There are two types of parity:

- Even Parity: The total number of '1' bits in the word must be even.
- Odd Parity: The total number of '1' bits in the word must be odd.

Considering an 8-bit word, 7 bits represent data, and the 8th bit is the parity bit. The following table presents examples of words including the parity bit:

7 bit data	Count of 1's	8 bit word	
		Even parity	Odd parity
0000000	0	0000000**0**	0000000**1**
1001001	3	1001001**1**	1001001**0**
1100110	4	1100110**0**	1100110**1**
1111111	7	1111111**1**	1111111**0**

Figure: 1 Requeriments_ParityBit

The implemented circuit must meet the following specifications:

- It must have 7 data bits as input, using the FPGA switches to represent the data bits.
- It must include an input to select the parity type:
    - "0" for even parity
    - "1" for odd parity

The circuit output must be an 8-bit word (the 7 data bits plus the parity bit). The 8 bits will be displayed using LEDs.

1.- Design a circuit to determine the parity of the input data.
2.- Design a circuit to determine the parity bit, depending on the input data parity and the selected parity type.
3.- Implement the circuit on the FPGA.

- Use ***switches SW[1] to SW[7] for data input.***
- Use ***switch SW[17] to select the parity type.***
- Use ***LED LEDR[0] for the parity bit and LEDs LEDR[1] to LEDR[7] to display the input data.***

2.- FUNCTIONAL DESCRIPTION

The project was created using 7 XOR logic gates, with 2 defined inputs: a 7-bit buffer [0..6] and an additional input for the 8th bit, which is the parity bit. Corresponding LEDs were connected to display the outputs. To simplify the circuit, a bus was used for the buffer, and the LEDs were also connected in a lower section. The logical gates was selected in primitive logical.

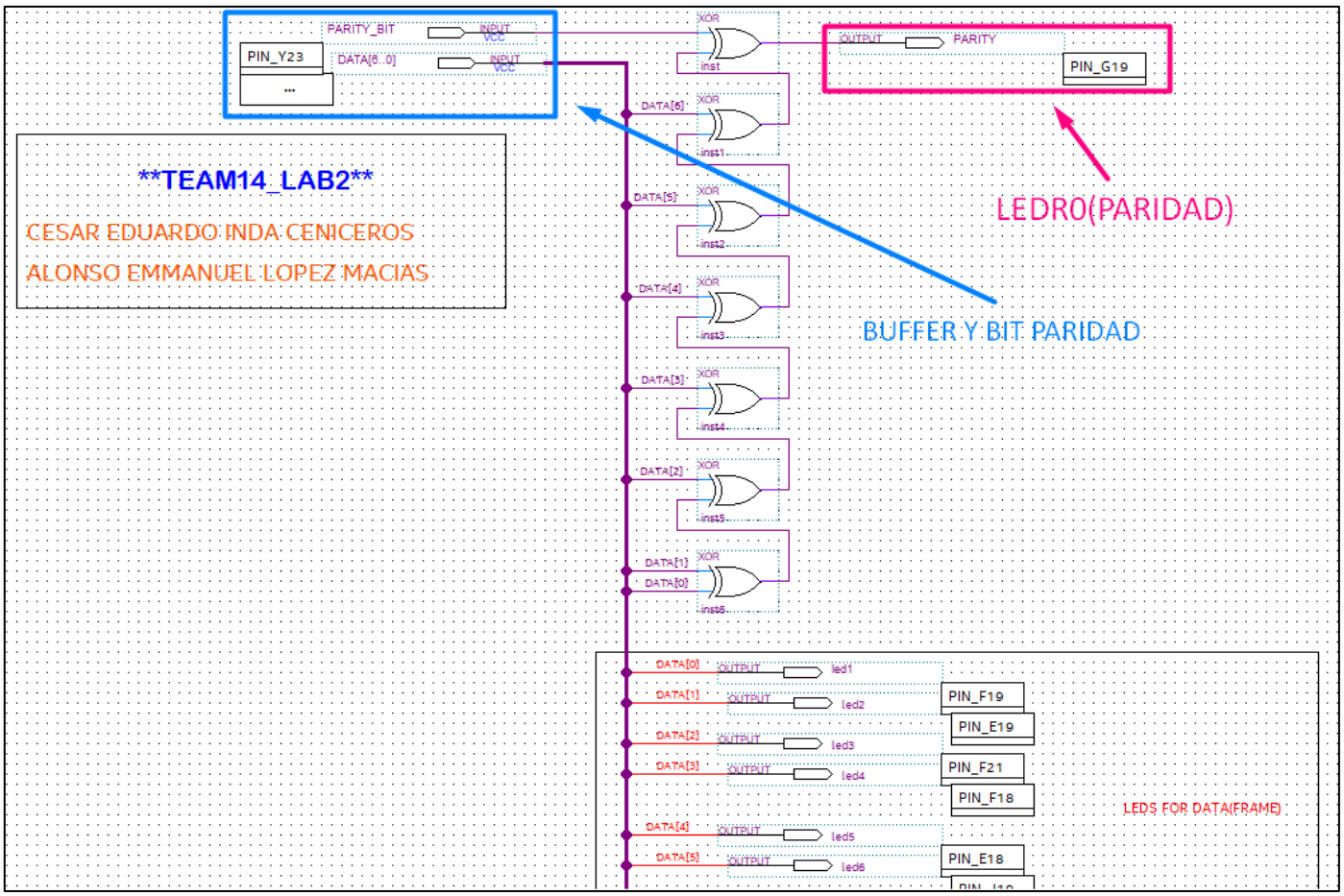


Figure: 2 ParityBit_Circuit

The pin assignment in the “Pin Planner” was as follows:

Node Name	Direction	Location
DATA[6]	Input	PIN_AB26
DATA[5]	Input	PIN_AD26
DATA[4]	Input	PIN_AC26
DATA[3]	Input	PIN_AB27
DATA[2]	Input	PIN_AD27
DATA[1]	Input	PIN_AC27
DATA[0]	Input	PIN_AC28
led1	Output	PIN_F19
led2	Output	PIN_E19
led3	Output	PIN_F21
led4	Output	PIN_F18
led5	Output	PIN_E18
led6	Output	PIN_J19
led7	Output	PIN_H19
PARITY	Output	PIN_G19
PARITY_BIT	Input	PIN_Y23

Figure: 3 PinPlanner

RTL VIEWER

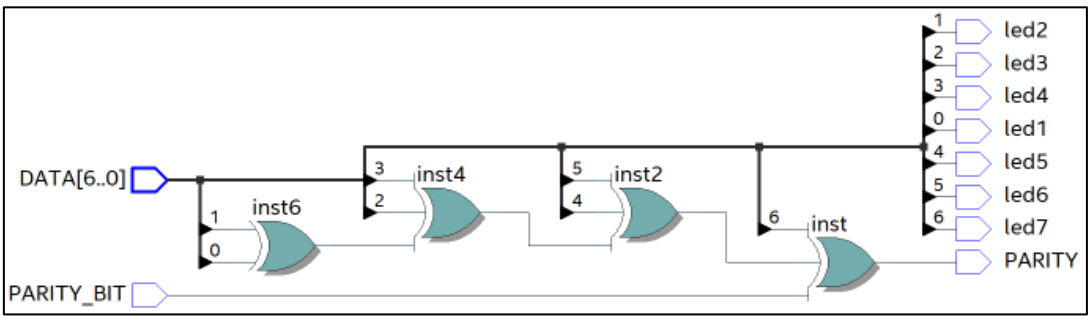


Figure: 4 RTL_Viewver

2.1.- Boolean Algebra Equations

For define the expressions was necessary first understand the begin.

Operation XOR between 2 values is: $X \oplus Y = \bar{X}Y + X\bar{Y}$

1.- A ⊕ B

Equation1: T1 = $\bar{A}B + A\bar{B}$

2.- (A ⊕ B) ⊕ C.

The definition with T1 is Equation2: $T2 = (\bar{A}B + A\bar{B}) \oplus C = [(\bar{A}B + A\bar{B})\bar{C}] + [(\bar{A}B + A\bar{B})\bar{C}]$

3.- Canonic Form: Sum Of Minterms With and Odd Numbers of 1’s

$$\left(\frac{7}{1}\right) + \left(\frac{7}{3}\right) + \left(\frac{7}{5}\right) + \left(\frac{7}{7}\right) = 7 + 35 + 21 + 1 = 64\ Impar\ Combinations$$

The equation is: $P = \Sigma(mi.) \oplus S$

mi = Minterms where exist an impar “1” numbers.

The result extended will be:

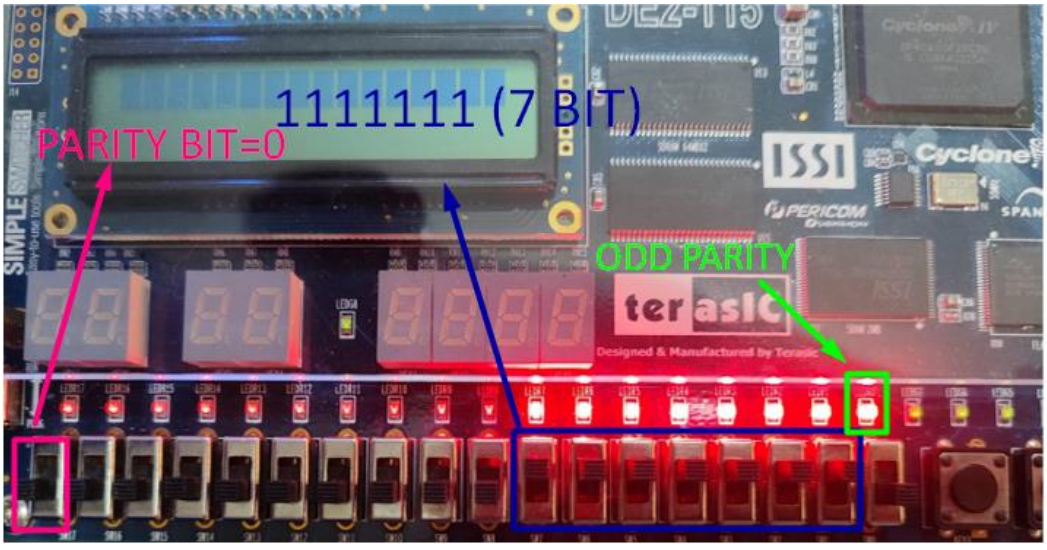
$P = A \oplus B \oplus C \oplus D \oplus E \oplus F \oplus G$

2.2.- Results

The truth table for this circuit would be as follows: In this case we have a brief from the table because there are 128 combinations of all the values.

D7	D6	D5	D4	D3	D2	D1	D0	Parity input	Error
1	0	1	1	0	0	1	0	1	1
1	1	0	0	1	0	0	0	1	0
1	0	1	1	1	0	1	1	1	1
1	0	1	1	1	1	1	0	0	0
0	0	1	0	1	0	1	0	1	0
0	1	1	1	0	1	0	1	0	1
0	1	0	1	0	0	1	1	1	1

Figure: 5 TrueTable_ParityBit

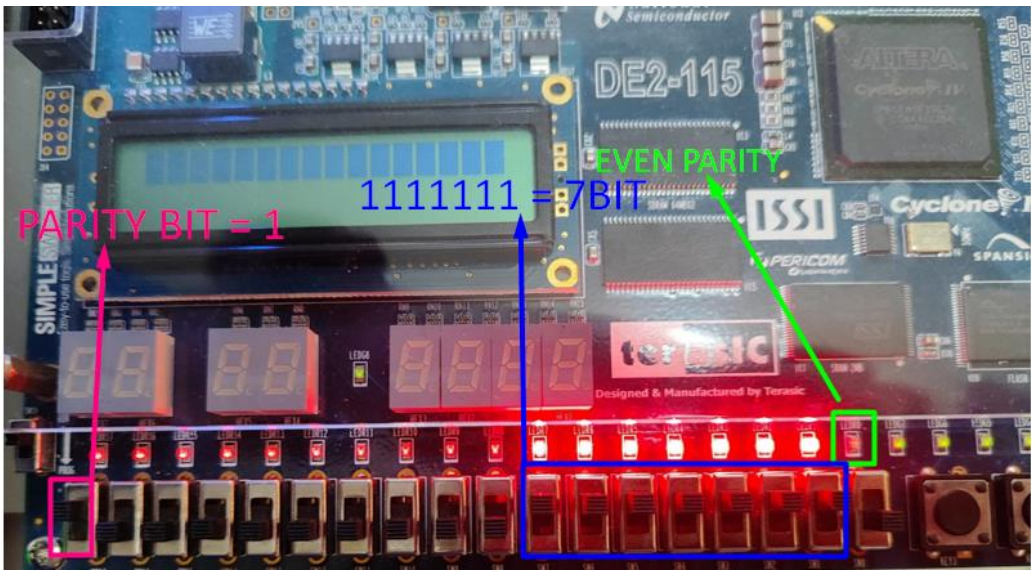


DATA = 1111111

PARITYBIT= 0

LEDR0 = 1

Figure: 6 Test1



DATA = 1111111

PARITYBIT= 1

LEDR0 = 0

Figure: 7 Test2

2.3.- GitHub Repository

A branch was created in the course repository corresponding to Lab2_ParityBit.

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA/commits/QuartusLab2_ParityBit/

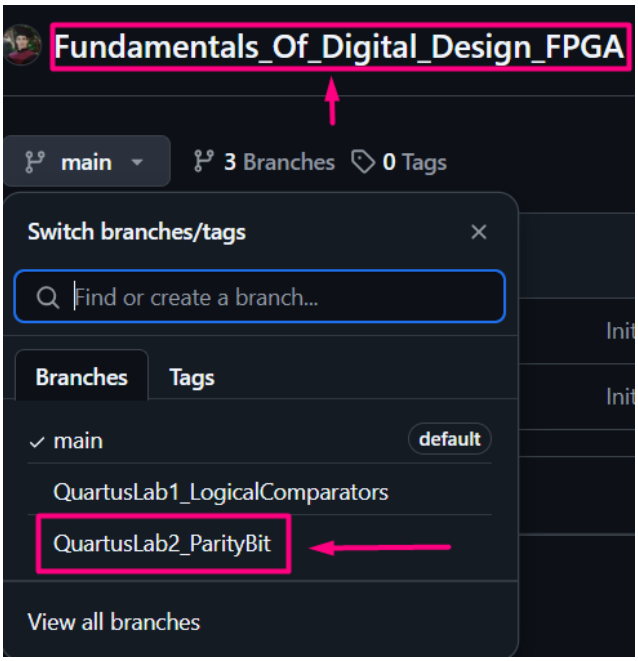


Figure: 8 GitHub Repository

3.- CHALLENGUES/ISSUES

Alonso Emmanuel López Macías

One of the main complications was understanding the circuit logic, since the XOR gate had one of its inputs defined by the output of the other gate, as if it were a cascading or progressive connection. From my perspective, analyzing the circuit and interpreting the logic to place the logic gates was somewhat complicated. In particular, when assigning pins, we had initially considered having two outputs referenced to the same state bit — whether even or odd — which ultimately led us to understand that it was actually the same bit. The difference was that when a logical input of 1 was entered, it would be reflected at the output, and if that logical 1 wasn't entered, it wouldn't appear.

Cesar Eduardo Inda Ceniceros

One complication we faced was understanding that it was a combinational logic system — meaning that many of these values are stored based on their previous logical state, resembling memory behavior. This also posed a challenge in understanding the circuit logic, particularly in defining the XOR gates in our case, along with their input and output parameters. A large part of this logic was defined by the equations we had previously generated, which allowed us to correctly assign the pins in the layout. Another important point was understanding when we were starting the value on our bus using SW0 or SW1, since we defined a data bus as [6..0], which also includes the value 0. We were often more accustomed to interpreting it as [7..1].

4.- CONCLUSIONS

Alonso Emmanuel López Macías

Personally, this practice allowed me to understand in greater detail the importance of the parity bit and how it influences communications in any electronic device. It helped me grasp the concept beyond simply determining whether a message or data is even or odd — it represents the importance of being able to detect errors in any transmission, such as in the case of an FPGA.

Cesar Eduardo Inda Ceniceros

Personally, this practice allowed me to understand in greater detail the importance of the parity bit and how it influences communications in any electronic device. It helped me grasp the concept beyond simply determining whether a message or data is logical or odd — it represents the importance of being able to detect errors in any transmission, such as in the case of an FPGA. This mechanism of detecting errors is very important to ensure proper implementation of security and validation.